

An abstract composition of various geometric shapes and patterns. In the top left, a green right-angled triangle points towards the top right. To its right is a blue semi-circle. Below the green triangle is a blue circle. In the center is a large orange semi-circle. To the right of the orange semi-circle are two vertical yellow bars. In the bottom left is a large orange circle. Above it are two yellow curved lines. To the right of the large orange circle are three yellow curved lines of different lengths and orientations. In the bottom right is a green square frame. The background is white.

1

Outline

- Terminology
 - Signature
 - Formal vs. Actual
 - Parameter Correspondence
- Passing Modes
 - Semantic
 - Actual
- Passing Arrays

Terminology

Prototype/Declaration

Portion of subprogram header that tells us its interface but NOT its implementation

Contains:

1. Return type (if any)
2. Name
3. *Number, Order, and Type* of all Parameters

The type signature OMITS the return type

Parameters

Within a definition:

```
int magic(int a, int b, int c) {  
    return b * b - 4 * a * c;  
}
```

Within a call:

```
int d = magic(a, a + 1, a * 4);
```

Parameters

Within a definition:

Called a **formal parameter**

- A “dummy” variable listed in the signature
- Used within the function’s implementation

Within a call:

Called an **actual parameter**

- Represents a value or address
- Determined at the point of execution



Parameter Correspondence

Actual Parameters must map to **Formal Parameters**

How do you think we should be able to map these?



Parameter Correspondence

Positional

- First actual parameter corresponds to first formal parameter, second actual parameter corresponds to second formal parameter, ...
- Can make correspondence mistakes with long lists
- Easy to check/implement

Keyword

- The name associated with the formal parameter must be used at the call site
- Parameters can appear in any order: `f (a3 = "hi", a2 = 10, a1 = True)`
- Intent can be clearer, but it's harder to check/implement

A decorative yellow dashed line is positioned on the left side of the slide, curving around the top-left corner of the white circle. A solid blue circle is located at the bottom-right edge of the white circle.

Passing Modes

Semantic Models

- In mode

Information is sent TO the subprogram

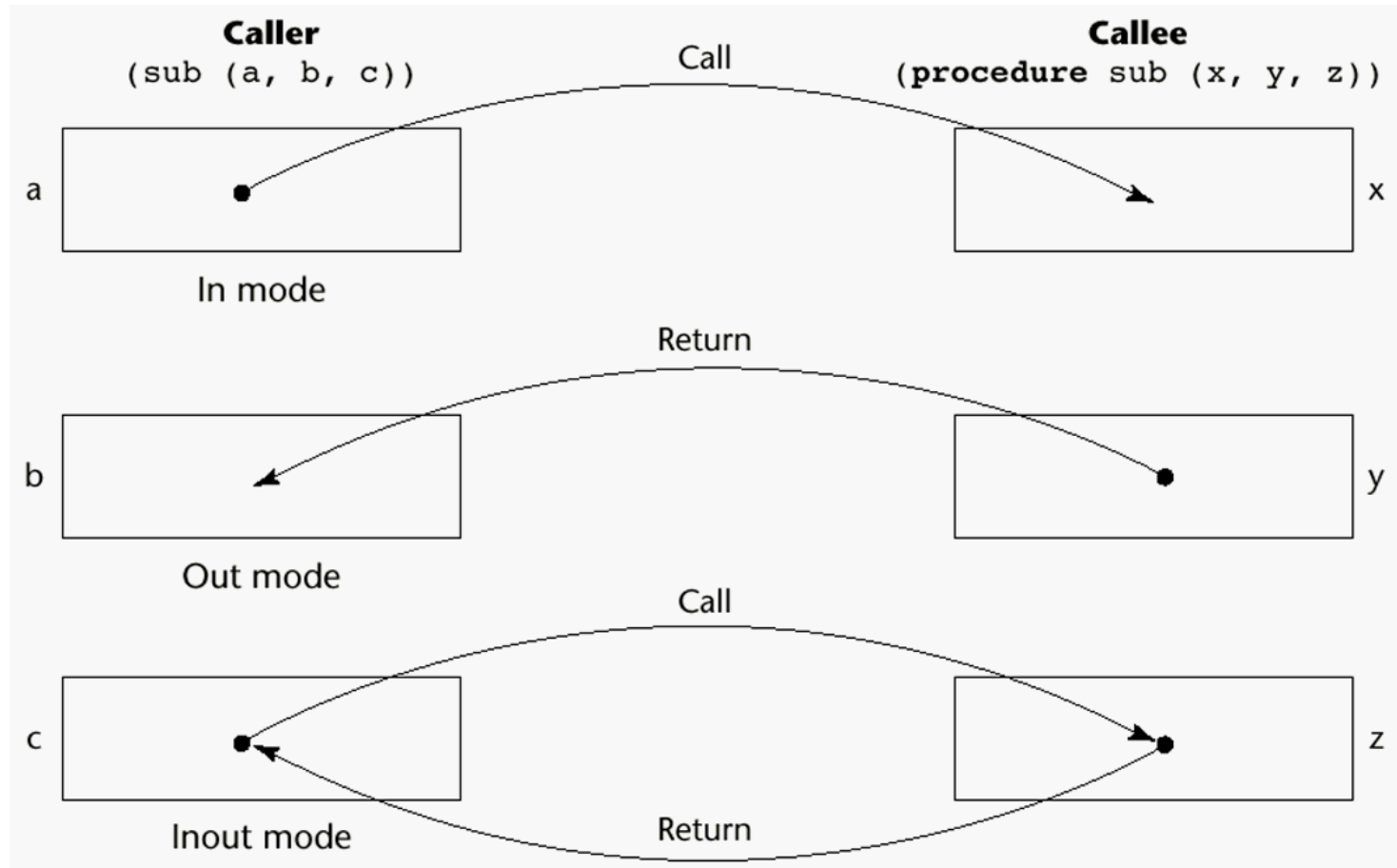
- Out mode

Information is retrieved FROM the subprogram

- In-Out mode

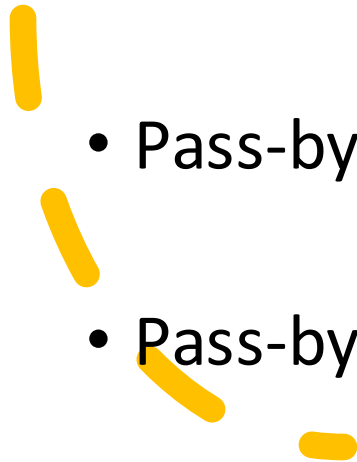
Information is (sent/received) (to/from) the subprogram

Semantic Models





Implementation Models

- Pass-by-Value
 - Pass-by-Result
 - Pass-by-Value-Result
 - Pass-by-Reference
 - Pass-by-Name
- 

Pass-by-Value (In mode)

- **Value** of the actual parameter is used to initialize its corresponding formal parameter
- Underlying value is usually *copied* from one memory cell to another
- *Advantages:*
 - Simple
- *Disadvantages:*
 - Additional storage required
 - Copied value can be large (in space)

Pass-by-Value

```
int by_value (int x) {  
    int y = 4;  
    x += 4;  
    return y + x;  
}  
  
int y = 3;  
int z = by_value (y);  
// y =  
// z =
```

Pass-by-Value

Most languages do pass-by-value as default:

- C/C++
- Java
- C#
- JavaScript
- Python

Pass-by-Result (Out mode)

- There is NO passed initial value, but the formal parameter acts like a local variable
- Parameters does not receive the initial value of the argument. Only gets a value at the end of the function and returns to caller.
- *Advantages:*
 - Useful for when you have multiple “outputs”
- *Disadvantages:*
 - For `fun(x, list[x])` should the address of `list[x]` be determined before or after execution?
 - Special semantics need to be determined for calls that use the same memory cell twice: `fun(a, a)`

Pass-by-Result

```
void by_result (int x) {  
    // int y = x + 2; // Error!  
    int y = 2;  
    x = y + 4;  
}
```

```
int y = 3;  
by_result (y);  
// y =
```

Pass-by-Result (C# Example)

```
static void Main()
{
    double radiusValue = 3.92781;
    CalculateCircumferenceAndArea(radiusValue,
        out double circumferenceResult, out var areaResult);
    System.Console.WriteLine($"Circumference of a circle with a radius of
{radiusValue} is {circumferenceResult}.");
    System.Console.WriteLine($"Area of a circle with a radius of
{radiusValue} is {areaResult}.");
    Console.ReadLine();
}

public static void CalculateCircumferenceAndArea(double radius,
        out double circumference, out double area)
{
    circumference = 2 * Math.PI * radius;
    area = Math.PI * (radius * radius);
}
```

Pass-by-Value-Result (In-Out mode)

- Expected combination of pass-by-value and pass-by-result
- Also called pass-by-copy (copy ALL THE THINGS)
- Formal parameters all have local storage

Pass-by-Value-Result

```
void by_value_result (int x) {  
    int y = x;  
    x = y + 4;  
}
```

```
int y = 3;  
by_value_result (y);  
// y =
```

Pass-by-Value-Result

Very few languages use pass-by-value-result:

- Fortran

Pass-by-Reference (In-Out mode)

- Instead of passing a value, pass an access path (i.e. memory address)
- Also called pass-by-sharing
- *Advantages:*
 - Lower memory footprint
- *Disadvantages:*
 - Slower access (must dereference to retrieve value)
 - Potential side effects (multiple reference updates)
 - Aliasing decreases readability

Pass-by-Reference

```
void by_reference (int x) {  
    int y = x;  
    x = y + 4;  
}
```

```
int y = 3;  
by_reference (y);  
// y =
```

Pass-by-Reference

Most languages can use pass-by-reference:

- C++ (use of &)
- C# (**ref** keyword)
- ~~Java (reference semantics for objects)~~
- ~~Python* (for mutable types)~~

Pass-by-Name (In-Out mode)

- Literal text substitution, i.e., it is re-evaluated every time it is used inside the function
- Formals bound at time of call. Values bound at time of reference or assignment
- *Advantages:*
 - Provides the latest possible binding
 - Extremely flexible
- *Disadvantages:*
 - No obvious semantics by looking at it

*It was possible to do this natively in ALGOL;
harder to do today due to language design decisions.*

Pass-by-Name (Jensen's Device)

```
#define SUM(Type, Var, Low, High, Term) \  
({ \  
    int low = (Low); \  
    int high = (High); \  
    Type sum = (Type)0; \  
    for (int Var = low; Var <= high; ++Var) { \  
        sum += (Term); \  
    } \  
    sum;  
})
```

Term behaves like a function of Var
and is re-evaluated on each use

Pass-by-Name (Jensen's Device)

```
double sum = SUM(double, i, 1, 100, 1.0 / i);
```

// expands to

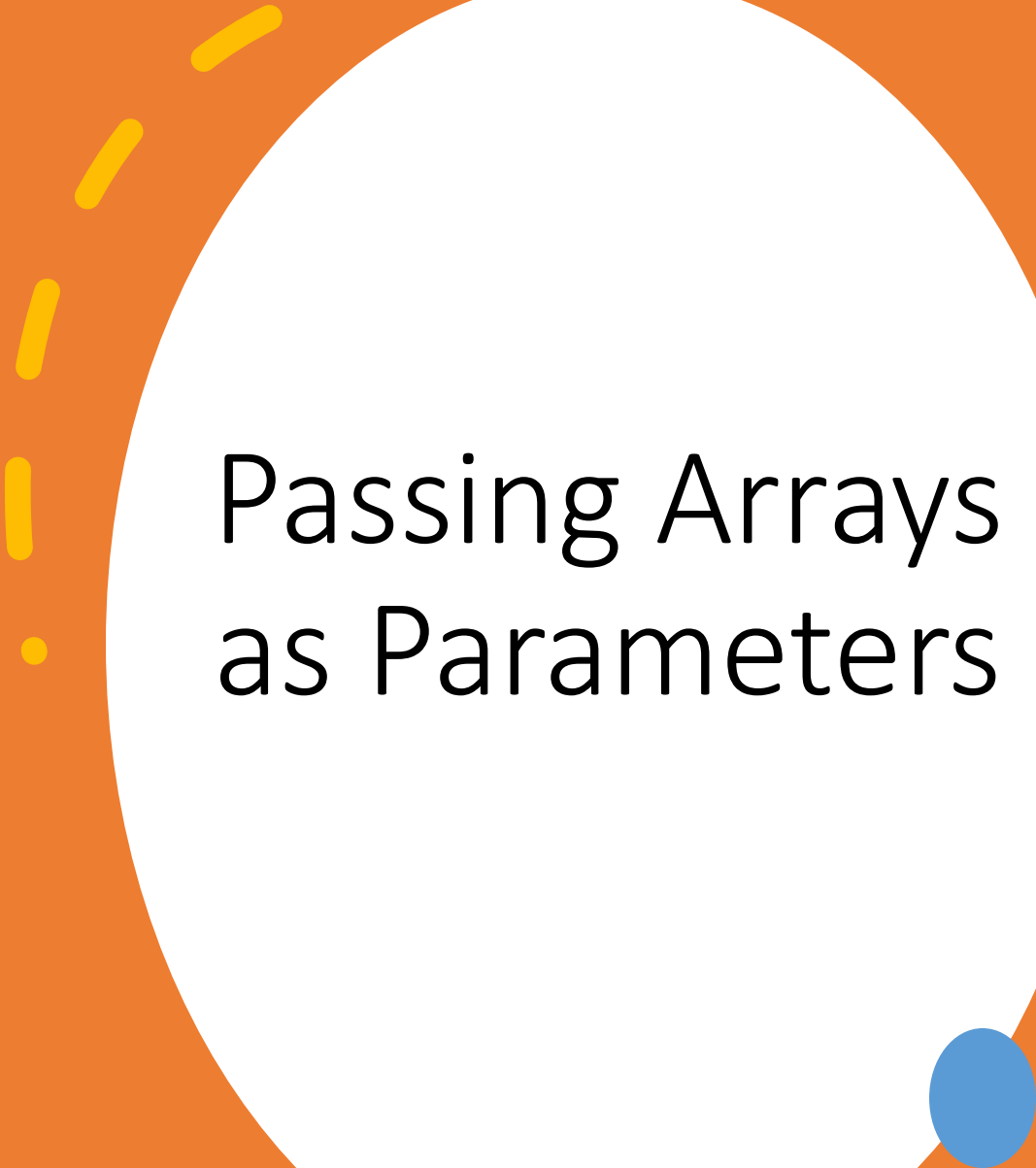
```
double sum = ({  
    int low = (1);  
    int high = (100);  
    double sum = (double)0;  
    for (int i = low; i <= high; ++i) {  
        sum += (1.0 / i);  
    }  
    sum;  
});
```

Pass-by-Name (Jensen's Device)

```
double prod = SUM(double, i, 1, 100,  
                  SUM(double, j, i, 100, i * j));
```

// expands to

```
double prod = ({  
    int low = (1);  
    int high = (100);  
    double sum = (double)0;  
    for (int i = low; i <= high; ++i) {  
        sum += ({  
            int low = (i);  
            int high = (100);  
            double sum = (double)0;  
            for (int j = low; j <= high; ++j) {  
                sum += (i * j);  
            }  
            sum;  
        }));  
    }  
    sum;  
});
```

A decorative yellow dashed line curves from the top left towards the center, ending in a small yellow dot. A solid blue circle is positioned at the bottom right edge of the white oval.

Passing Arrays as Parameters

Languages Where Passing Arrays is Fine and Works as Expected

C#

D

Java

Python

JavaScript

Ruby

Perl

PHP

Lisp

OCaml

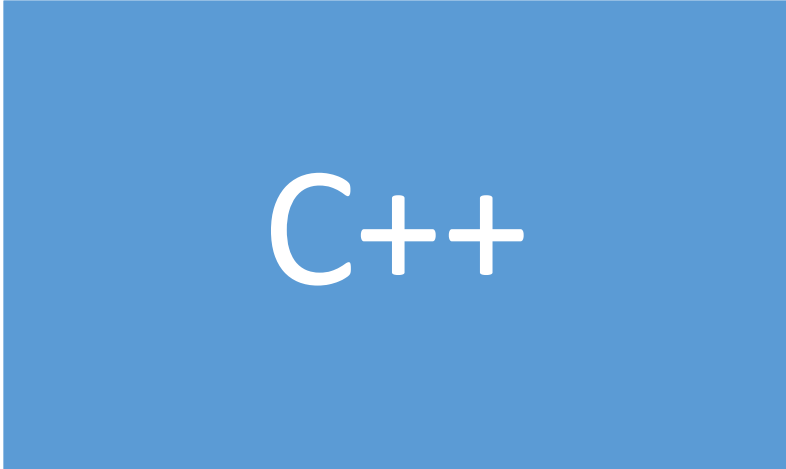
F#

Fortran

Languages Where Passing Arrays is Weird



C



C++

Array Passing in C/C++

Definition of a static or fixed stack array in C:

```
int arr[5];
```

Type of `arr` is `int[5]`

Passing `arr` as a parameter will decay its type.

Decay means coercion with loss of information.

```
int*
```

Array Passing in C/C++

Definition of a fixed heap-dynamic array in C++:

```
int* arr = new int[5];
```

The type of `arr` is `int*`

Passing `arr` as a parameter doesn't change its type:

```
int*
```

Array Passing in C/C++

We don't want heap-based and non-heap-based arrays to interact differently, so they need to be converted to a common type

- Though it is possible to accept only fixed arrays!

Multi-Dimensional Array Passing in C++

```
double m[3][4];
```

```
// valid
```

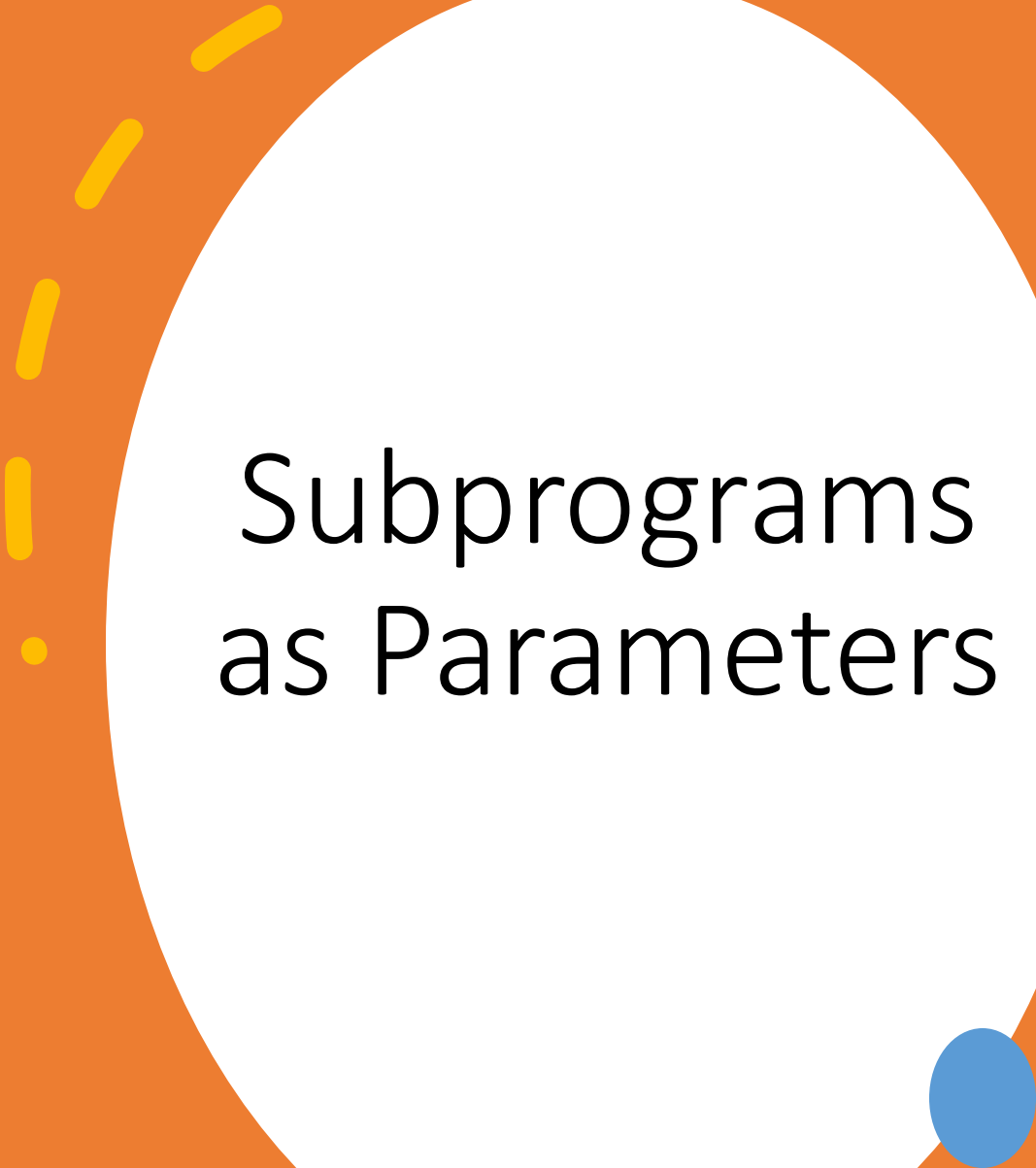
```
void mat_inverse (double (&m)[3][4])
```

```
void mat_inverse (double* m[4])
```

```
// invalid
```

```
void mat_inverse (double** m)
```

Decay on outer-most dimension **only**

A decorative yellow dashed line curves from the top left towards the center, ending in a small yellow dot. A solid blue circle is positioned at the bottom right edge of the white oval.

Subprograms as Parameters

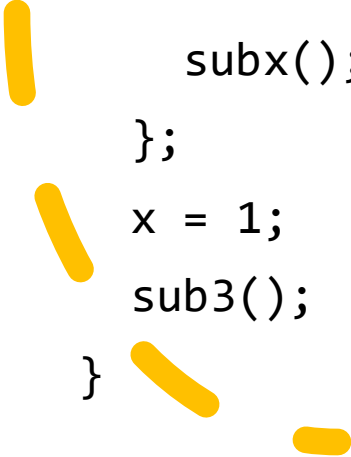
Referencing Environment

How do we resolve non-local references for subpgm. passed as param?

- *Shallow binding*: the environment when the function is called
 - Most natural for dynamic-scoped languages
- *Deep binding*: the environment when the function is passed
 - Most natural for static-scoped languages
- *Ad hoc binding*: the environment where the function was defined



```
function sub1() {  
  var x;  
  function sub2() {  
    alert(x);  
  };  
  function sub3() {  
    var x = 3;  
    sub4(sub2);  
  };  
  function sub4(subx) {  
    var x = 4;  
    subx();  
  };  
  x = 1;  
  sub3();  
}
```



Shallow: x = ?

Deep: x = ?

Ad hoc: x = ?

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz

modified by Gary Zoppetti

modified by Jingnan Xie

Sources include Robert Sebesta's [Concepts of Programming Languages](#)